# Visual Recognition using Deep Learning

Homework 2: Digit Detection Problem

Muhammad Rayhan Athaillah

Student ID: [Your ID]

National Yang Ming Chiao Tung University

April 17, 2026

**GitHub Repository:** https://github.com/RayhanHaqi/[Your-Repo]

## 1 Introduction

This report outlines the technical implementation for the Digit Detection Problem (Homework 2) within the Visual Recognition using Deep Learning course[cite: 4, 10]. The objective is to design a robust object detection pipeline capable of identifying and localizing multiple digits in varied RGB images[cite: 145, 148].

### 1.1 Technical Constraints and Requirements

The assignment dictates a specific model architecture: the system must utilize the **DEtection TRansformer (DETR)** as the core model, paired exclusively with a **ResNet-50** backbone[cite: 163]. While the backbone is fixed, modifications to other components—such as the transformer encoder and decoder—are permitted[cite: 164]. The following constraints are strictly enforced:

- **Data Usage:** The use of external data from any outside source is strictly prohibited[cite: 165].
- **Pretrained Weights:** Pretrained weights are allowed only for the ResNet-50 backbone[cite: 164].
- **Submission Standards:** Final deliverables must include a ZIP file containing the source code (.py) and a PDF report written in English[cite: 21, 271, 277]. Inclusion of checkpoints or datasets in the submission is forbidden[cite: 279, 280].

- **Academic Integrity:** A zero-tolerance policy for plagiarism is in effect, with severe penalties including failure of the course[cite: 287, 288].

# 2 Literature Review

## 2.1 Architectural Evolution: From DETR to RT-DETRv2

The emergence of the DEtection TRansformer (DETR) introduced a paradigm shift by utilizing a bipartite matching loss and a transformer-based encoder-decoder to eliminate the need for hand-crafted components like Non-Maximum Suppression (NMS)[cite: 163, 164]. As proposed by Carion et al. [1], DETR frames detection as a direct set prediction problem [1].

Figure 1: Architecture of the original DEtection TRansformer (DETR) [1].

Real-Time DETR (RT-DETR) later improved upon this by introducing an efficient hybrid encoder to handle multi-scale features, allowing the model to achieve speeds comparable to YOLO architectures while maintaining transformer accuracy [2] [2]. The latest iteration, **RT-DETRv2**, further optimizes this baseline by refining the backbone interaction and training stability, which is highly effective for dense detection tasks [3] [3].

Figure 2: The RT-DETRv2 architecture optimized for real-time performance [3].

## 2.2 Optimization and Regularization Strategies

Deep transformer training typically benefits from the **AdamW** optimizer [4], which improves generalization by decoupling weight decay from the gradient updates [4]. Within the DETR loss function, the **End-of-Sequence (`eos_coef`)** coefficient is vital for penalizing "no-object" (background) predictions. Tuning this penalty is essential to manage false positives and accommodate potential labeling noise in complex datasets.

## 2.3 Resolution Adaptation

Analysis of the provided dataset indicates a predominantly widescreen aspect ratio. Consequently, the input resolution is adapted to $320 \times 640$ to align with the RT-

DETRv2 architecture's requirements (multiples of 32) while preserving the natural geometry of the digits.

# 3 Methodology

## 3.1 Preprocessing and Resolution Check

The initial preprocessing phase involved auditing the dataset resolution. To maintain spatial features without vertical distortion, images were resized to $320 \times 640$ pixels. This rectangular input ensures the model learns the correct aspect ratios of the digits.

## 3.2 Data Augmentation

To improve robustness without shifting bounding box coordinates, pixel-level augmentations were employed:

- **ColorJitter:** Random adjustment of brightness, contrast, and saturation.
- **RandomErasing:** Strategically masking image patches to simulate oclusions.

## 3.3 Regularization and Training Scheme

Advanced regularization was implemented to ensure stable convergence:

- **LR Scheduler:** The **OneCycleLR** scheduler was used to navigate the optimization landscape efficiently.
- **EMA: Exponential Moving Average** of model weights was utilized to enhance the stability of final predictions.

## 3.4 Hyper-parameter Experiments

Systematic experiments were conducted using a Dual NVIDIA RTX 4090 setup, focusing on the following variables:

- **Object Queries ($q$):** Evaluation of $q = 1000$ versus $q = 1500$ to accommodate digit density.
- **Weight Decay (WD):** Comparative testing between $1 \times 10^{-3}$ and $5 \times 10^{-4}$ for optimal regularization.
- **Learning Rate (LR):** Tuning the base learning rate at $1 \times 10^{-4}$ to balance speed and accuracy.

# References


[1] M. Carion et al., "End-to-End Object Detection with Transformers," in *Proc. ECCV*, 2020.

[2] W. Lv et al., "DETRs Beat YOLOs on Real-time Object Detection," in *Proc. CVPR*, 2023.

[3] W. Lv et al., "RT-DETRv2: Improved Baseline with Faster Training and Better Generalization," *arXiv:2407.17140*, 2024.

[4] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019.